PHASE 7 — PQC RECOMMENDATION ENGINE & CRYPTOGRAPHIC MIGRATION DECISION SYSTEM

Project: Enterprise Cryptographic Discovery & Analysis Tool (ECDAT)

Problem Statement: SIH26164

Organization: National Technical Research Organisation (NTRO)

Theme: Blockchain & Cybersecurity

Phase: 7 of 22

Document Type: Technical Research & Engineering Handbook

Status: Recommendation & Migration Intelligence Foundation

Table of Contents

22. When Hybrid Makes Sense
23. When Hybrid Does Not Make Sense
24. Algorithm Selection Factors
25. Security
26. Performance
27. Key Size
28. Ciphertext Size
29. Signature Size
30. Latency
31. Memory
32. CPU
33. Bandwidth
34. Hardware Compatibility
35. HSM Compatibility
36. PKI Compatibility
37. TLS Compatibility
38. Cloud Compatibility
39. Application Compatibility
40. Embedded-System Compatibility
41. Vendor Constraints
42. Regulatory Constraints
43. Crypto-Agility
44. Migration Complexity
45. Multi-Objective Optimization
46. Recommendation Scoring
47. Hard Constraints
48. Soft Constraints
49. Recommendation Confidence
50. Primary Recommendation
51. Alternative Recommendations
52. Rejected Recommendations
53. Recommendation Explanation
54. Example 1 — ECDSA

---

1. Purpose of This Phase

The previous phases established:

```
Phase 1
Cryptography
```

```
        ↓
    Phase 2
    Quantum Threat
        ↓
    Phase 3
    PQC
        ↓
    Phase 4
    Enterprise Crypto Ecosystem
        ↓
    Phase 5
    Discovery + CBOM
        ↓
    Phase 6
    Quantum Risk Assessment
```

We now have a difficult problem.

ECDAT knows:

```
WHAT EXISTS
```

and:

```
WHAT IS RISKY
```

But NTRO explicitly requires:

> **Recommend suitable alternatives (PQC/Hybrid algorithms) for applications based on risk profile, latency, cost, etc.**

That means ECDAT must answer:

"What should this specific system use instead?"

This is the purpose of Phase 7.

---

2. Where We Are in the ECDAT Pipeline

The overall system now looks like:

```
                ENTERPRISE
                    |
                    ▼
                DISCOVERY
```

```
                    |
                    ▼
                  CBOM
                    |
                    ▼
              RISK ENGINE
                    |
                    ▼
          PQC RECOMMENDATION
                    |
                    ▼
            MIGRATION PLAN
```

Phase 7 is the decision layer between:

```
Risk
```

and:

```
Migration
```

---

3. The Core Recommendation Problem

Imagine ECDAT discovers:

```
Algorithm:
ECDSA P-256

Function:
Digital Signature

Application:
Payment Gateway

Criticality:
Critical

Latency Requirement:
Very Low

Hardware:
HSM
```

A naive system might say:

> Replace ECDSA with ML-DSA.

That is not enough.

ECDAT must ask:

- Does the TLS stack support it?
- Does the HSM support it?
- Do clients support it?
- What is the certificate strategy?
- What is the signature size?
- What is the performance impact?
- Is hybrid deployment appropriate?
- What migration path minimizes operational risk?

Therefore:

> **Recommendation is a constrained engineering decision, not a simple lookup table.**

---

4. Why Simple Algorithm Replacement Fails

Consider:

```
RSA
↓
ML-KEM
```

This looks reasonable.

But RSA may be serving as:

- Encryption
- Key transport
- Digital signature
- Certificate key
- Code signing

ML-KEM is a KEM.

It is not a universal RSA replacement.

Similarly:

```
ECDSA
↓
ML-KEM
```

is incorrect because ECDSA is a signature algorithm.

The appropriate replacement class must preserve the original cryptographic function.

Therefore:

```
FUNCTION
      ↓
REPLACEMENT FAMILY
      ↓
ALGORITHM
```

is mandatory.

---

5. Recommendation as a Decision Problem

ECDAT can model recommendation as:

```
Find algorithm A
such that:

Security(A)
is acceptable

AND

Compatibility(A)
is acceptable

AND

Performance(A)
meets requirements

AND
```

In mathematical terms, conceptually:

```
A* = argmax Utility(A)
```

subject to organizational and technical constraints.

The exact optimization model can evolve later.

---

6. Inputs to the Recommendation Engine

The engine should consume:

Cryptographic context

```
Algorithm
Function
Parameters
Protocol
Library
```

Risk context

```
Quantum Risk
Business Criticality
Data Lifetime
Exposure
```

Engineering context

```
Latency
CPU
Memory
Bandwidth
Storage
```

Infrastructure context

```
HSM
KMS
TLS stack
Certificate infrastructure
Hardware
```

Organizational context

```
Policy
Compliance
```

```
Budget
Migration deadline
Vendor support
```

---

7. Cryptographic Function Classification

Before recommending anything, ECDAT should determine what the current algorithm is actually doing.

Possible functions:

```
Key Encapsulation
Key Exchange
Digital Signature
Encryption
Authentication
Hashing
Integrity
Key Wrapping
Certificate Signing
Code Signing
Firmware Signing
```

Example:

```
ECDSA
```

becomes:

```
Function:
Digital Signature
```

Then:

```
Candidate Family:
PQC Signature
```

---

8. KEM Recommendation

If the existing mechanism is used for key establishment, ECDAT should consider a KEM.

Conceptually:

```
Classical KEX
       ↓
PQC KEM
```

For example:

```
ECDH
↓
ML-KEM
```

During transition:

```
ECDH
+
ML-KEM
```

may be evaluated where the relevant protocol supports a hybrid construction.

---

9. Digital Signature Recommendation

If the existing mechanism is:

```
RSA Signature
ECDSA
EdDSA
```

the recommendation engine should evaluate PQC signature schemes.

Potential standardized candidates include:

```
ML-DSA
SLH-DSA
```

The choice depends on:

- Ecosystem support
- Performance
- Signature size
- Security assumptions
- Hardware support
- Operational constraints

---

10. Symmetric Cryptography Recommendations

PQC is not simply:

```
Replace every algorithm.
```

For symmetric cryptography, the strategy is different.

For example:

```
AES-256-GCM
```

may remain appropriate.

A quantum-risk engine should understand the distinction between:

```
Public-Key Quantum Vulnerability
```

and:

```
Symmetric Quantum Security Margin
```

Therefore, ECDAT must avoid indiscriminate replacement recommendations.

---

11. Hash Function Recommendations

Similarly, hash functions require contextual evaluation.

Example:

```
SHA-256
```

could be used for:

- Integrity
- Password hashing
- Digital signatures
- Merkle trees
- Content addressing

The migration decision depends on the function.

A hash algorithm appearing in source code does not automatically mean:

> "Replace it with another hash."

---

12. Legacy Algorithm Mapping

ECDAT should maintain a conceptual mapping:

```
Legacy Algorithm
        ↓
Function
        ↓
Candidate PQC Family
        ↓
Candidate Algorithms
        ↓
Compatibility Analysis
```

Example:

```
ECDH
 ↓
Key Establishment
 ↓
PQC KEM
 ↓
ML-KEM
 ↓
Protocol / Hardware Check
```

---

13. RSA Decision Logic

RSA can perform different roles.

RSA Key Transport

Potential direction:

```
RSA
↓
KEM
```

RSA Signature

Potential direction:

```
RSA
↓
```

```
PQC Signature
```

RSA Certificate

Potential direction:

```
PQC-capable PKI
```

RSA Code Signing

Potential direction:

```
PQC Signature
```

Therefore ECDAT must identify the role before recommending a replacement.

---

14. ECC Decision Logic

ECC may include:

```
ECDSA
ECDH
EdDSA
```

Each must be treated separately.

```
ECDH
↓
KEM migration

ECDSA
↓
Signature migration

EdDSA
↓
Signature migration
```

This function-aware mapping is essential.

---

15. DH Decision Logic

Traditional Diffie-Hellman relies on discrete logarithm assumptions.

ECDAT can classify:

```
DH
↓
Quantum Vulnerable
↓
Key Establishment
↓
PQC KEM / Hybrid Candidate
```

Again, the exact migration path depends on protocol support.

---

16. Weak Symmetric Algorithm Decision Logic

Legacy algorithms may require immediate classical remediation regardless of quantum risk.

Examples:

```
3DES
Weak configurations
Obsolete modes
```

ECDAT should distinguish:

```
Classical Security Remediation
```

from:

```
Quantum Migration
```

A system can have both risks simultaneously.

---

17. Weak Hash Decision Logic

Similarly:

```
SHA-1
```

may already be unsuitable for certain security functions.

ECDAT should therefore support:

```
Classical Risk
+
Quantum Risk
```

rather than assuming quantum risk is the only concern.

---

18. ML-KEM

ML-KEM is a key candidate for post-quantum key establishment.

Common standardized parameter sets include:

```
ML-KEM-512
ML-KEM-768
ML-KEM-1024
```

ECDAT should not simply select the largest parameter set.

It should evaluate:

```
Security Requirement
+
Performance
+
Bandwidth
+
Memory
+
Protocol Support
```

---

19. ML-DSA

ML-DSA is a standardized lattice-based signature mechanism.

Common parameter sets include:

```
ML-DSA-44
ML-DSA-65
ML-DSA-87
```

Selection must consider:

- Required security strength
- Signature size
- Signing speed
- Verification speed
- Hardware

- Ecosystem support

---

20. SLH-DSA

SLH-DSA provides a hash-based signature alternative.

It offers algorithmic diversity relative to lattice-based signatures.

That diversity matters.

An organization might prefer to avoid having every critical system rely on exactly the same mathematical assumption.

ECDAT should therefore support:

```
Lattice-based
```

and:

```
Hash-based
```

recommendation paths.

---

21. Hybrid Cryptography

Hybrid cryptography combines classical and post-quantum mechanisms during migration.

Conceptually:

```
Classical
   +
PQC
   ↓
Combined Security
```

For example:

```
ECDH
+
ML-KEM
```

can be considered for suitable key-establishment protocols.

The exact combination must follow standardized protocol constructions.

---

22. When Hybrid Makes Sense

Hybrid migration can be useful when:

- Legacy clients still exist
- PQC support is emerging
- Compatibility matters
- Organizations want defense in depth
- Migration needs to happen gradually

Example:

```
2026
Classical

2027–2029
Hybrid

2030+
PQC-primary
```

This is only an illustrative migration pattern.

---

23. When Hybrid Does Not Make Sense

Hybrid is not automatically better.

It may introduce:

- Increased complexity
- Larger messages
- More implementation surface
- More dependencies
- Performance overhead

Therefore ECDAT should not recommend hybrid simply because:

> "Hybrid is safer."

It must evaluate whether hybrid is technically appropriate.

---

24. Algorithm Selection Factors

Candidate algorithms should be evaluated across:

```
Security
Performance
Size
Latency
Memory
CPU
Bandwidth
Hardware
Protocol
Cloud
PKI
Vendor
Cost
Migration Complexity
```

---

25. Security

The first filter is security.

ECDAT should prioritize:

1. Standardized algorithms
2. Well-analyzed implementations
3. Appropriate parameter sets
4. Current security guidance

Experimental algorithms should not automatically receive production recommendations.

---

26. Performance

Performance metrics:

```
Key Generation
Encapsulation
Decapsulation
Signing
Verification
```

ECDAT should consider application-specific performance requirements.

Example:

```
High-frequency API
↓
Latency-sensitive
```

versus:

```
Offline archive
↓
Latency-insensitive
```

---

27. Key Size

Larger public keys can affect:

- Certificates
- Handshakes
- Storage
- Memory

ECDAT should record exact sizes for candidate algorithms.

---

28. Ciphertext Size

For KEMs, ciphertext size matters.

Large ciphertexts can affect:

```
TLS
VPN
API traffic
Embedded systems
Low-bandwidth links
```

Therefore ECDAT should include ciphertext size in its optimization model.

---

29. Signature Size

Signature size can become significant in:

```
Software distribution
Firmware
Blockchain
```

```
Certificates
High-volume transactions
```

Therefore signature size should influence recommendations.

---

30. Latency

A real-time system might have strict requirements:

```
Maximum latency:
5 ms
```

A candidate algorithm with much higher overhead may not be suitable.

ECDAT should therefore allow:

```
Latency Budget
```

as an input.

---

31. Memory

Embedded systems may have:

```
RAM:
256 KB
```

A PQC implementation that requires substantially more memory may be infeasible.

Therefore:

```
Memory Constraint
```

should be a hard filter for some assets.

---

32. CPU

PQC algorithms may have different computational profiles.

ECDAT should consider:

```
CPU Architecture
CPU Capacity
Hardware Acceleration
```

```
Workload
Request Volume
```

---

33. Bandwidth

This becomes important for:

- IoT
- Satellites
- Mobile networks
- VPN
- Large-scale APIs

An algorithm with larger cryptographic messages may create significant network overhead.

---

34. Hardware Compatibility

ECDAT should identify:

```
CPU
Operating System
HSM
TPM
Smart Card
Network Appliance
```

and determine whether the selected cryptographic implementation is supported.

---

35. HSM Compatibility

For enterprise systems, this can become a hard constraint.

Example:

```
Application
↓
HSM
↓
Private Key
```

If the HSM does not support the selected PQC signature mechanism, the recommendation may need to change.

Possible output:

```
Primary:
ML-DSA

Constraint:
Current HSM lacks production support.

Action:
Upgrade HSM firmware / model
before migration.
```

This is much more useful than blindly recommending an algorithm.

---

36. PKI Compatibility

PQC migration can require:

- Certificate Authority changes
- Certificate profile changes
- Client support
- Validation library support
- Trust-store changes

ECDAT should therefore evaluate the entire certificate chain.

---

37. TLS Compatibility

The recommendation engine should understand:

```
TLS version
Client population
Server implementation
Cipher suite
Key exchange
Certificate
```

A PQC algorithm that cannot be deployed in the organization's current TLS ecosystem may require a staged strategy.

---

38. Cloud Compatibility

Cloud providers may have varying support for:

- PQC
- KMS
- Certificates
- TLS policies
- HSMs

ECDAT should maintain a compatibility knowledge base.

Conceptually:

```
Algorithm
↓
Cloud Provider
↓
Service
↓
Support Status
```

---

39. Application Compatibility

The recommendation engine should consider application architecture.

Examples:

```
Monolith
Microservice
Serverless
Mobile
Embedded
Desktop
```

A server-side application is not equivalent to an embedded device.

---

40. Embedded-System Compatibility

Embedded systems may have:

```
Low memory
Low CPU
Limited storage
Slow networks
```

```
Long lifecycle
Rare updates
```

Therefore algorithm selection may be constrained by hardware.

Migration may require:

```
Firmware update
```

or:

```
Hardware refresh
```

---

41. Vendor Constraints

Organizations may depend on vendors.

Example:

```
Firewall Vendor
↓
PQC support unavailable
```

ECDAT should flag:

```
Migration Blocker:
Vendor support required
```

rather than falsely claiming migration is immediately possible.

---

42. Regulatory Constraints

Some environments have strict requirements.

Examples:

- Government
- Banking
- Defense
- Healthcare
- Critical infrastructure

ECDAT should allow policy modules.

For example:

```
Policy:
Only approved cryptographic algorithms
may be used for production.
```

The exact approved list must be configurable and based on authoritative organizational requirements.

---

43. Crypto-Agility

A highly crypto-agile system can support:

```
Algorithm A
↓
Algorithm B
```

without major application redesign.

A rigid system might require:

```
Hardware Replacement
+
Software Rewrite
+
Protocol Redesign
```

Therefore crypto-agility should strongly influence migration recommendations.

---

44. Migration Complexity

ECDAT should estimate:

```
Low
Medium
High
Very High
```

based on:

```
Code changes
Protocol changes
Infrastructure changes
Certificate changes
Hardware changes
```

```
Testing effort
Vendor dependencies
Downtime
```

---

45. Multi-Objective Optimization

Suppose three candidates exist.

Candidate	Security	Performance	Size	Compatibility
A	High	High	Small	Medium
B	Very High	Medium	Large	High
C	High	Very High	Medium	Low

There is no universally best choice.

ECDAT must optimize according to the application's constraints.

---

46. Recommendation Scoring

A conceptual scoring model:

```
Recommendation Score =
Security Weight
+
Performance Weight
+
Compatibility Weight
+
Migration Weight
+
Cost Weight
```

More rigorously:

```
Score(A)
=
```

```
Σ wi × normalized_metric_i(A)
```

subject to hard constraints.

The weights should be configurable.

---

47. Hard Constraints

Some requirements should eliminate a candidate entirely.

Example:

```
Required:
HSM support

Candidate:
No HSM support
```

Then:

```
Candidate = REJECTED
```

Another:

```
Memory budget:
512 KB

Candidate requirement:
2 MB

Candidate = REJECTED
```

This is better than merely assigning it a lower score.

---

48. Soft Constraints

Other factors can influence ranking without eliminating a candidate.

Example:

```
Preference:
Low bandwidth
```

Candidate A:

```
Low bandwidth
```

Candidate B:

```
Moderate bandwidth
```

Both remain valid, but A ranks higher.

---

49. Recommendation Confidence

ECDAT should output:

```
Recommendation:
ML-DSA

Confidence:
High
```

or:

```
Recommendation:
ML-DSA

Confidence:
Medium

Reason:
HSM compatibility not yet verified.
```

This prevents false certainty.

---

50. Primary Recommendation

The output should contain:

```
PRIMARY RECOMMENDATION
```

Example:

```
ML-DSA
```

with:

```
Why:
Best fit for signature function,
supported security requirements,
acceptable performance,
and organizational policy.
```

---

51. Alternative Recommendations

The engine should provide alternatives.

Example:

```
Primary:
ML-DSA

Alternative:
SLH-DSA

Reason:
Algorithmic diversity / different security assumption.
```

This is valuable for strategic planning.

---

52. Rejected Recommendations

One of the most powerful explainability features:

```
Not Recommended:
Algorithm X

Reason:
HSM incompatibility

Not Recommended:
Algorithm Y

Reason:
Exceeds latency budget
```

This shows the system actually evaluated alternatives.

---

53. Recommendation Explanation

A good recommendation should answer:

What?

```
ML-DSA
```

Why?

```
Signature function
+
PQC requirement
```

Why now?

```
Mosca risk
+
Long-lived data
```

Why this algorithm?

```
Compatibility
+
Performance
+
Security
```

What must change?

```
PKI
HSM
TLS
Clients
```

---

54. Example 1 — ECDSA

Input:

```
Algorithm:
ECDSA P-256

Function:
TLS Signature
```

```
Criticality:
High

Data Lifetime:
15 years

HSM:
Supported only through current vendor stack

Latency:
Strict
```

Possible output:

```
Primary:
ML-DSA

Transition:
Evaluate hybrid/staged deployment

Migration:
High

Dependencies:
TLS
PKI
HSM
Client compatibility

Confidence:
Medium-High
```

---

55. Example 2 — RSA Key Exchange

Input:

```
RSA
Function:
Key Establishment
```

ECDAT should recognize:

```
Not a signature problem.
```

Then evaluate:

```
PQC KEM
```

Potential direction:

```
ML-KEM
```

with hybrid options where the protocol supports them.

---

56. Example 3 — Code Signing

Input:

```
ECDSA P-256
Function:
Firmware Signature
```

ECDAT considers:

```
Signature Algorithm
+
Firmware Size
+
Bootloader
+
Hardware
+
Update Mechanism
+
Device Lifecycle
```

Potential recommendation:

```
PQC Signature
```

but migration may require firmware and bootloader updates.

---

57. Example 4 — Legacy VPN

Input:

```
IPsec
DH Group
Classical
```

ECDAT evaluates:

```
Key Establishment
↓
PQC KEM / Hybrid
```

Then checks:

```
VPN gateway
Firmware
Peer compatibility
Bandwidth
Performance
```

Potential output:

```
Hybrid migration recommended
```

if supported by the deployed ecosystem.

---

58. Example 5 — Embedded Device

Input:

```
Device:
Industrial Controller

RAM:
256 KB

CPU:
Low-power

Update:
Rare

Lifecycle:
15 years
```

ECDAT should not simply choose the strongest algorithm.

It must optimize:

```
Security
+
Memory
+
CPU
+
Firmware Size
+
Update Mechanism
```

Potential recommendation:

```
PQC candidate
+
Firmware migration plan
+
Hardware validation
```

---

59. Example 6 — Cloud Application

Input:

```
Cloud:
Managed Kubernetes

TLS:
Managed ingress

Keys:
Cloud KMS
```

ECDAT checks:

```
Cloud KMS support
Ingress support
Certificate service
Client compatibility
```

If PQC cannot yet be fully deployed:

```
Recommendation:
Prepare crypto-agility
+
Hybrid-compatible architecture
+
Monitor provider support
```

---

60. Migration Sequencing

Not everything should migrate simultaneously.

ECDAT should create:

```
Migration Wave 1
Critical + High Risk

Migration Wave 2
Important Business Systems

Migration Wave 3
Medium Risk

Migration Wave 4
Low Risk / Legacy Cleanup
```

---

61. Migration Dependency Graph

Example:

```
HSM Upgrade
      ↓
CA Upgrade
      ↓
Certificate Migration
      ↓
TLS Migration
      ↓
Application Migration
```

ECDAT should identify this dependency order.

Otherwise an organization may attempt to migrate an application before the underlying infrastructure is ready.

---

62. Migration Waves

A realistic plan might be:

```
WAVE 0
Inventory + Architecture

WAVE 1
Non-production pilots

WAVE 2
Low-risk production

WAVE 3
Medium-risk systems

WAVE 4
Critical systems

WAVE 5
Legacy retirement
```

---

63. Pilot Deployment

Before large-scale migration:

```
Select Pilot
↓
Enable PQC
↓
Measure
↓
Test Compatibility
↓
Monitor
↓
Adjust
```

Pilot systems should ideally represent realistic workloads.

---

64. Testing Strategy

PQC migration requires multiple forms of testing.

Functional

Does authentication work?

Performance

Is latency acceptable?

Interoperability

Do clients and servers communicate?

Security

Does implementation behave correctly?

Failure

What happens when PQC negotiation fails?

Rollback

Can the system safely return to the previous configuration?

---

65. Performance Validation

ECDAT should recommend measurement.

Metrics:

```
Handshake latency
CPU utilization
Memory
Network overhead
Throughput
Request latency
Certificate size
Signature verification
```

This turns recommendation into engineering evidence.

---

66. Interoperability Testing

A PQC algorithm may be supported by one component but not another.

Example:

```
Client:
Supports PQC

Server:
Supports PQC

Load Balancer:
Doesn't
```

The migration fails.

ECDAT should map the complete communication chain.

---

67. Rollback Strategy

Migration should include:

```
Deployment
↓
Monitoring
↓
Failure Detection
↓
Rollback
```

This is particularly important for critical systems.

---

68. Migration Readiness Gates

ECDAT can define gates:

Gate 1

Inventory complete.

Gate 2

Risk assessed.

Gate 3

PQC algorithm selected.

Gate 4

Compatibility validated.

Gate 5

Performance validated.

Gate 6

Security testing complete.

Gate 7

Production migration approved.

---

69. Recommendation Policies

Organizations may want rules such as:

```
For Critical Systems:
PQC migration planning mandatory.
```

or:

```
No new quantum-vulnerable
cryptographic deployment.
```

or:

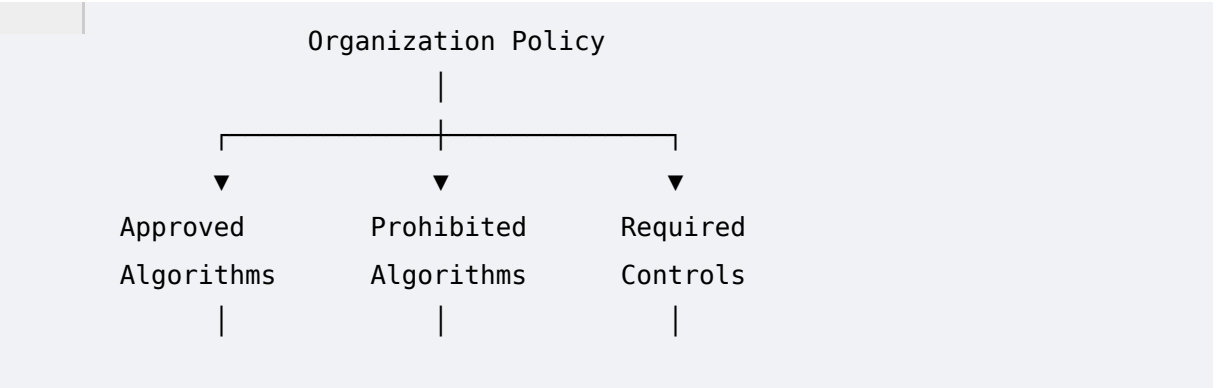
```
Use only approved standardized
PQC algorithms.
```

These become policy constraints.

---

70. Organizational Policy Engine

Conceptually:



```
                └──────────┬──────────┘
                           ▼
                    Recommendation
```

This makes ECDAT customizable.

---

71. Example Policy

```
POLICY: PQC-001

Scope:
Critical production systems

Requirement:
New cryptographic deployments must
support a documented PQC migration path.

Allowed:
Standardized PQC algorithms

Exception:
Legacy systems with documented waiver

Review:
Annual
```

The recommendation engine can enforce this.

---

72. AI's Role

AI can assist with:

Context understanding

```
"Payment authorization service"
```

Code interpretation

```
CustomCryptoProvider
```

Documentation analysis

```
Migration dependencies
```

Recommendation explanation

```
Why this algorithm?
```

Migration-plan generation

```
What should the engineering team do?
```

But AI should not independently override deterministic security policies.

---

73. Human-in-the-Loop

For high-impact decisions:

```
ECDAT
↓
Recommendation
↓
Security Architect Review
↓
Approval
↓
Migration
```

This is especially important for:

- Critical infrastructure
- Government systems
- HSM changes
- Root CA migration
- Firmware trust anchors

---

74. Recommendation Auditability

Every recommendation should preserve:

```
Input
↓
Evidence
↓
```

```
Constraints
↓
Candidates
↓
Scoring
↓
Recommendation
```

Example:

```
Recommendation ID:
REC-002918

Asset:
CRYPTO-00129

Selected:
ML-DSA

Rejected:
Candidate X — HSM unsupported
Candidate Y — latency exceeds limit

Confidence:
High

Generated:
2026-08-24
```

---

75. Recommendation Versioning

The recommendation may change over time.

Example:

```
2026:
ML-DSA recommended

2028:
New standardized option available

2030:
Vendor support changes
```

Therefore ECDAT should preserve historical recommendations.

---

76. Algorithm Lifecycle

The algorithm knowledge base should track:

```
Experimental
↓
Candidate
↓
Standardized
↓
Production Recommended
↓
Deprecated
↓
Discouraged
↓
Broken
```

This lifecycle must be continuously updated.

---

77. Handling Future PQC Algorithms

ECDAT should be designed so new algorithms can be added without rewriting the entire engine.

For example:

```
Algorithm Registry
        │
        ├── ML-KEM
        ├── ML-DSA
        ├── SLH-DSA
        └── Future Algorithm
```

Each algorithm should have metadata:

```
Family
Function
Security
Performance
Size
```

```
Compatibility
Status
```

---

78. Recommendation Engine Architecture

A conceptual architecture:

```
                    CBOM
                     │
                     ▼
               RISK ENGINE
                     │
                     ▼
          FUNCTION CLASSIFIER
                     │
                     ▼
           CANDIDATE GENERATOR
                     │
                     ▼
            HARD CONSTRAINTS
                     │
                     ▼
          ┌──────────────────┐
          │ Candidate Set    │
          └──────────────────┘
```

---

79. Complete Decision Pipeline

The full ECDAT decision process:

```
DISCOVER
   ↓
IDENTIFY
   ↓
CLASSIFY
   ↓
ASSESS RISK
   ↓
IDENTIFY FUNCTION
   ↓
GENERATE PQC CANDIDATES
   ↓
```

```
CHECK HARD CONSTRAINTS
   ↓
EVALUATE PERFORMANCE
   ↓
CHECK COMPATIBILITY
```

---

80. Requirements Derived From Phase 7

ECDAT now requires a dedicated recommendation subsystem.

Algorithm Knowledge Base

Must store:

- Algorithm
- Function
- Family
- Security
- Parameter sets
- Performance
- Sizes
- Compatibility
- Lifecycle
- Status

Recommendation Engine

Must support:

- Function-aware mapping
- Candidate generation
- Hard constraints
- Soft constraints
- Multi-objective ranking
- Policy enforcement
- Confidence
- Alternatives

Migration Engine

Must support:

- Dependency ordering

- Migration waves
- Pilot planning
- Testing
- Rollback
- Readiness gates

AI

Must support:

- Semantic interpretation
- Context enrichment
- Explanation
- Migration-plan assistance

---

81. Phase 7 Summary

Phase 7 establishes one of the most important principles of ECDAT:

> **There is no universal "best PQC algorithm."**

The appropriate algorithm depends on:

```
Function
+
Security
+
Performance
+
Size
+
Latency
+
Hardware
+
Protocol
+
PKI
+
HSM
+
```

Therefore ECDAT must function as a **decision-support system**, not an algorithm lookup table.

---

82. The Core Intelligence

A basic system says:

```
ECDSA
↓
ML-DSA
```

A mature ECDAT system says:

```
ECDSA P-256
|
├── Function: Signature
├── Protocol: TLS
├── Criticality: High
├── Data Lifetime: 15 years
├── HSM: Vendor X
├── Latency: Strict
├── Client Ecosystem: Mixed
└── Crypto-Agility: Medium
        |
        ▼
Candidate Analysis
        |
        ├── ML-DSA
        ├── SLH-DSA
        └── Hybrid / staged strategy
        |
```

That is the level of intelligence NTRO's problem statement is implicitly asking for.

---

83. Phase 7 Final Takeaway

The ECDAT recommendation engine should ultimately answer four questions:

1. What should replace this?

```
PQC / Hybrid candidate
```

2. Why?

```
Security + context + constraints
```

3. What prevents immediate migration?

```
Hardware
Protocol
Vendor
Client
Cost
```

4. What should we do next?

```
Pilot
↓
Validate
↓
Migrate
↓
Retire legacy
```

That makes the recommendation actionable.

PHASE 8 PREVIEW — AI-POWERED CRYPTOGRAPHIC CODE ANALYSIS

This is where the project starts getting **really interesting**.

So far, ECDAT can use deterministic discovery techniques.

But real-world cryptographic usage is often hidden behind abstractions.

Phase 8 will focus on the **AI layer**:

- Why regex alone fails
- AST analysis
- Call-graph analysis
- Data-flow analysis
- Semantic cryptography detection
- LLM-assisted code reasoning
- Cryptographic intent detection
- Detecting custom crypto wrappers

- Detecting indirect library usage
- Understanding configuration-driven cryptography
- Detecting algorithm selection at runtime
- Source-to-binary correlation
- AI confidence scoring
- Evidence-grounded AI
- Retrieval-Augmented Generation
- Local vs cloud models
- Model selection
- Prompt architecture
- Agentic scanning
- AI hallucination control
- Human verification
- Explainable findings
- AI red-team considerations
- Benchmarking AI detection accuracy
- False-positive reduction
- False-negative reduction
- Creating a cryptographic code corpus
- Training/evaluation datasets
- LLM + static-analysis hybrid architecture

And most importantly:

> **How do we use AI without allowing AI to make unsupported cryptographic claims?**

The architecture will evolve toward:

```
SOURCE / BINARY
      |
      ▼
STATIC ANALYSIS
      |
      ▼
DETERMINISTIC EVIDENCE
      |
      ▼
```

```
                    AI SEMANTIC LAYER
                          |
                          ▼
                  EVIDENCE CORRELATION
                          |
                          ▼
                   CONFIDENCE ENGINE
                          |
```

That is where we can start defining the **actual "crazy AI" component** of the SIH solution instead of simply putting an LLM chatbot on top of a scanner.

Phase 7 complete.